

CS3313计算理论 课程作业

uu-matter543

2025–2026–1

目录

1 Homework 01–Finite Automata 1	2
2 Homework 02–Finite Automata 2	3
3 Homework 03–Context Free 1	5
4 Homework 04–Context Free 2	7
5 Homework 05–Church Turing Thesis	8
6 Homework 06–Decidability	10
7 Homework 07–Reducibility 1	12
8 Homework 08–Reducibility 2	13
9 Homework 09–Time Complexity 1	15
10 Homework 10–Time Complexity 2	17
11 Homework 11–Space Complexity 1	20
12 Homework 12–Space Complexity 2	22
13 Homework 13–Space Complexity 3	23

1 Homework 01–Finite Automata 1

Problem 1 (1.4d)设计识别语言 $L = \{w | w \text{ has even } a, \text{ every } a \text{ is followed by at least } 1 b\}$ 的 DFA.

解 可根据正则语言交集的方法分步设计，如下列图所示.

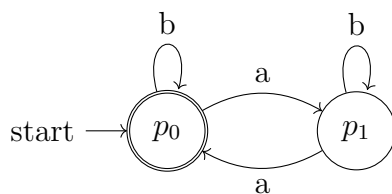


图 1: 识别 $\{w | w \text{ has even } a\}$ 的 DFA

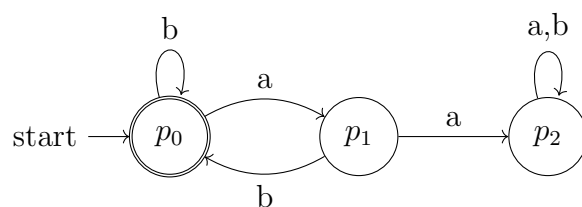


图 2: 识别 $\{w | \text{Every } a \text{ is followed by at least } 1 b\}$ 的 DFA

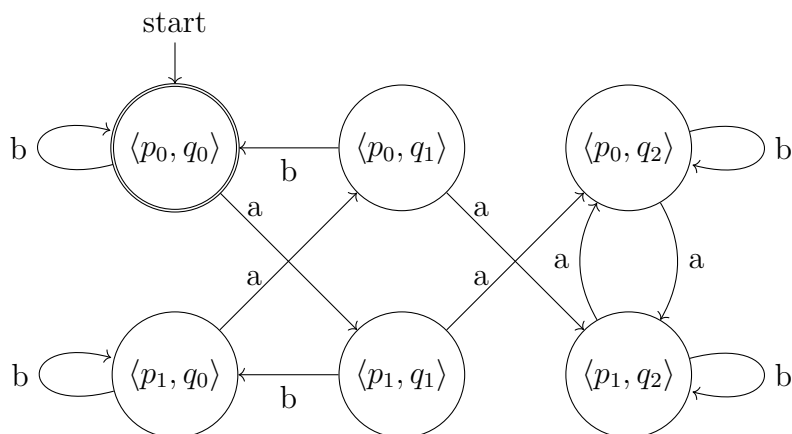


图 3: 识别 $\{w | w \text{ has even } a, \text{ every } a \text{ is followed by at least } 1 b\}$ 的 DFA

Problem 2 (1.11)证明: 所有 NFA 都可转化为等价的、只有唯一接受状态的 NFA.

解 对于任意 NFA $M = (Q, \Sigma, \delta, q_0, F)$, 添加状态 $q_{final} \in F$; 对 $\forall q \in F$, 将 q 移出 F , 并添加从 q 读取 ϵ 到 q_{final} 的转移边, 即可将 M 转化为题述要求的 NFA.

Problem 3 (1.31) 对于字符串 $w = w_1w_2w_3 \cdots w_n$, 记 $w^R = w_nw_{n-1}w_{n-2} \cdots w_2w_1$; 对语言 A , 记 $A^R = \{w^R | w \in A\}$. 证明: 若 A 是正则语言, 则 A^R 也是正则语言.

解 设 DFA $M = (Q, \Sigma, \delta, q_0, F)$ 识别 A , 可构造 NFA $N = (Q \cup \{q'_0\}, \Sigma, \delta', q'_0, \{q_0\})$ 识别 A^R , 其中 $\delta' = \delta^{-1} \cup \{f(\langle q'_0, \epsilon \rangle) = q | q \in F\}$, 则 $L(N) = A^R$, 所以 A^R 也是正则语言 (即: 将原 DFA 的状态转移边反向; 在新 NFA 中添加起始状态, 该起始状态可通过 ϵ 到达原 DFA 的任意接受状态; 新 NFA 的接受状态为原 DFA 的初始状态).

Problem 4 (1.38) 定义 all-NFA 为 5 元组 $M = (Q, \Sigma, \delta, q_0, F)$, M 接受 w 当且仅当 M 读取 w 后所有可能处于的状态 s 均满足 $s \in F$. 证明: all-NFA 识别的语言是正则语言 (注意: 我们通常定义的 NFA 的接受, 是指 NFA 读取完输入后, 部分可能处于的状态属于接受状态集而非全部).

解 分别证明充分性与必要性.

- 证明: 所有正则语言都能被某些 all-NFA 识别.

显然可以将 DFA 看作 all-NFA (因为: DFA 接受输入 $w \Leftrightarrow$ DFA 读取完输入后唯一处于的状态属于接受状态集), 即证.

- 证明: all-NFA 识别的语言是正则语言.

设 $N = (Q, \Sigma, \delta, q_0, F)$ 是 all-NFA, 构造 DFA $D = (P(Q), \Sigma, \delta', q'_0, F')$:

- $P(Q)$ 是 Q 的幂集; 对于 $R \in P(Q), a \in \Sigma, \delta'(R, a) = \bigcup_{r \in R} \{q | q = \delta(r, a)\}$;
- $q'_0 = \{q | q = \delta(q_0, \epsilon)\} \cup \{q_0\}$; $F' = \{R \in P(Q) | R \subseteq F\}$;

即: 在 NFA 重构为 DFA 的过程中, 将重构接受状态集步骤 $F' = \{R \in P(Q) | \exists q \in R, q \in F\}$ 改为 $F' = \{R \in P(Q) | \forall q \in R, q \in F\}$ (即对于新 DFA 中的状态, 其代表的原 all-NFA 中的状态均为原 all-NFA 的接受状态时, 才将其移入新 DFA 的接受状态集), 可保证 all-NFA 读取完某个输入后所有可能处于的状态都位于 DFA 的接受状态中, 因此 all-NFA 与对应的 DFA 等价, 即证.

2 Homework 02—Finite Automata 2

Problem 1 (1.29b) 利用泵引理证明: $\{www | w \in \{a, b\}^*\}$ 不是正则语言.

解 令 p 是泵临界值, $w = a^pb$, 则 $\because |xy| \leq p, \therefore \text{w.l.o.g. } x = a^m, y = a^n, z = a^{p-m-n}ba^pba^pb (1 \leq m, 1 \leq n, m+n \leq p)$, 此时 $xy^0z = a^{p-n}ba^pba^pb \notin \{www | w \in \{a, b\}^*\}$, 因此题述语言不是正则语言.

Problem 2 (1.47) $\Sigma = \{1, \#\}$ 上的语言 $Y = \{w | w = x_1 \# x_2 \# x_3 \# \cdots \# x_k, k \geq 0, i \neq j \Leftrightarrow x_i \neq x_j\}$, 证明: Y 不是正则语言.

解 $1^p \# 1^{p-1} \# 1^{p-2} \# \cdots \# 1 \# 1^0 \in Y$, $\because |uv| \leq p, \therefore uv \subset 1^p$,

w.l.o.g. $u = 1^m, v = 1^n (1 \leq n \leq p)$, 则 $uv^0w = 1^{p-n} \# 1^{p-1} \# 1^{p-2} \# \cdots \# 1 \# 1^0 \notin Y$ (因为 $p-n \in \{0, 1, 2, \dots, p-1\}$), 所以 Y 不可泵, 不是正则语言.

Problem 3 (1.49)

(1) 证明: $B = \{1^k y | k \geq 1, y \in \{0, 1\}^*, y \text{ has at least } k \text{ } 1\}$ 是正则语言;

(2) 证明: $C = \{1^k y | k \geq 1, y \in \{0, 1\}^*, y \text{ has at most } k \text{ } 1\}$ 不是正则语言.

解 (1) 对于 $w \in \Sigma_B^*$;

- 若 w 以 0 开头, 则不满足 $k \geq 1$, 此时 $w \notin B$;
- 若 w 以 1 个 1 开头, 则 $w \in B$ 当且仅当 w 除第 1 个 1 外至少包含 1 个 1;
- 若 w 以 2 个及以上 1 开头, 则令 $k = 1$ 可直接得出 $w \in B$;

根据以上分析可设计 DFA 如下:

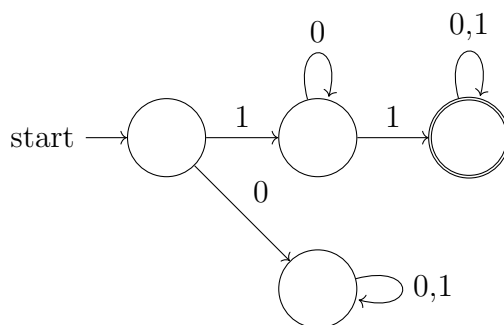


图 4: 识别 $B = \{1^k y | k \geq 1, y \in \{0, 1\}^*, y \text{ has at least } k \text{ } 1\}$ 的 DFA

因为 B 可被上述 DFA 识别, 所以 B 是正则语言.

(2) $1^p 0 1^p \in C$, $\because |uv| \leq p$, w.l.o.g. $u = 1^m, v = 1^n (1 \leq n \leq p)$, 则 $uv^2w = 1^{p+n} 0 1^p$, $p+n > p, uv^2w \notin C$, 所以 C 不可泵, 不是正则语言.

Problem 4 (1.54) $F = \{a^i b^j c^k | i, j, k \geq 0, i = 1 \Rightarrow j = k\}$.

(1) 证明 F 不是正则语言;

(2) 证明 F 满足泵引理;

(3) 说明为什么 (1) 和 (2) 与泵引理不矛盾.

解 (1)构造 $F_1 = \{b^j c^k | j, k \geq 0\}$, $F_2 = \{ab^j c^k | j \geq 0\}$, $F_3 = \{a^i b^j c^k | i \geq 2, j, k \geq 0\}$,
显然 $F = F_1 \cup F_2 \cup F_3$, 且 F_1, F_2, F_3 不相交, 所以 $F_2 = F - (F_1 \cup F_3) = F \cap \overline{F_1 \cup F_3}$,
则有: 根据正则语言在交、并、补操作下封闭, F 是正则语言 $\Rightarrow F_2$ 是正则语言,
根据反证法, 只要证明 F_2 不是正则语言即可证明 F 不是正则语言, 证明如下:

$$ab^p c^p \in F_2, \because |uv| \leq p, \therefore uv \subset ab^{p-1},$$

- $u = \epsilon, a \in v$, 由于 F_2 中的字符串均只包含 1 个 a , 因此 $uv^0 w \notin F_2$;
 - $u \neq \epsilon, w.l.o.g. u = ab^m, v = b^n (1 \leq n \leq p-1)$, 显然 $uv^0 w = ab^{p-n} c^p \notin F_2$;
- 因此 F_2 不可泵, 不是正则语言.

则 F 也不是正则语言.

(2)分类讨论如下:

- $i = 0, j = 0, k = 0$ 不满足 $|uvw| > p$, 不符合泵引理的前提;
- $i = 0, j = 0, k > 0$, 取 $u = \epsilon, v = c$ 即可泵;
- $i = 0, j > 0$, 取 $u = \epsilon, v = b$ 即可泵;
- $i = 1, j = k$, 取 $u = \epsilon, v = a$ 即可泵;
- $i = 2$, 取 $u = \epsilon, v = aa$ 即可泵, 因为此时 a 的个数必然为偶数, 不可能为 1, 对 b, c 的个数无要求, 符合 $a^i b^j c^k$ 的格式即可;
- $i \geq 3$, 取 $u = \epsilon, v = a$ 即可泵, 因为此时 a 的个数 ≥ 2 , 不可能为 1, 对 b, c 的个数无要求, 符合 $a^i b^j c^k$ 的格式即可;

综上所述, F 满足泵引理.

(3)可泵性只是正则语言的必要非充分条件, 非正则语言也可能满足泵引理.

3 Homework 03—Context Free 1

Problem 1 (2.4b)设计生成语言 $\{w \in \{0,1\}^* | w \text{ begins and ends with the same symbol}\}$ 的 CFG.

解 $S \rightarrow 0T0 | 1T1 | 0 | 1 \quad T \rightarrow 0T | 1T | \epsilon$

Problem 2 (2.6a)设计生成语言 $\{w \in \{a,b\}^* | w \text{ has more } a \text{ than } b\}$ 的 CFG.

解 S 生成 a 的数量至少比 b 的数量多 1 的串, T 生成 a 的数量与 b 相等的串.

$S \rightarrow aS|aT|bSS \quad T \rightarrow aTb|bTa|TT|\epsilon$

Problem 3 (2.10) 设计识别 $A = \{a^i b^j c^k | i = j \vee j = k, i, j, k \geq 0\}$ 的 PDA.

解 $S \rightarrow S_1 | S_2 \quad S_1 \rightarrow aS_1 | E_1 \quad E_1 \rightarrow bE_1 c | \epsilon \quad S_2 \rightarrow S_2 c | E_2 \quad E_2 \rightarrow aE_2 b | \epsilon$

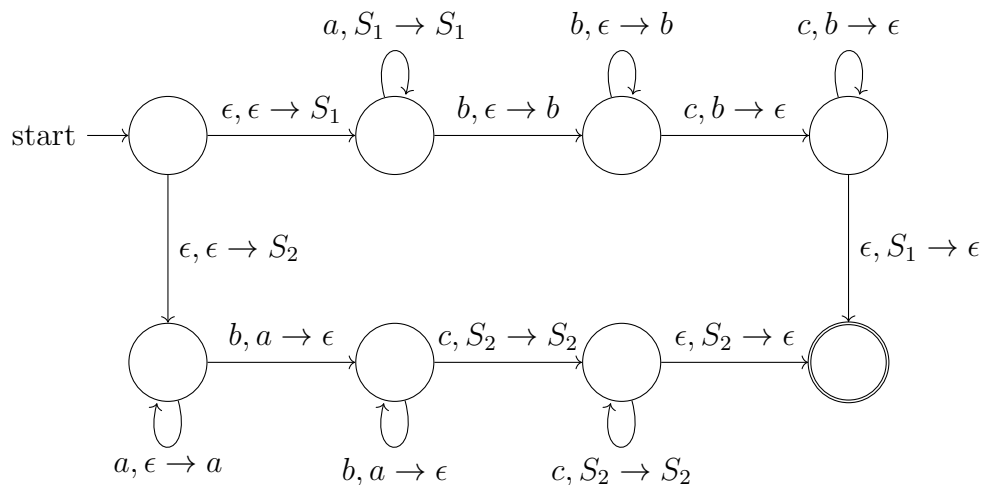


图 5: 识别 $A = \{a^i b^j c^k | i = j \vee j = k, i, j, k \geq 0\}$ 的 PDA

Problem 4 (2.26) 设 G 是乔姆斯基范式的 CFL, 证明: 从 G 生成得到 $w \in L(G)$ 需要 $2|w| - 1$ 次重写.

解 通过数学归纳法证明 (在如下的证明过程中, S 为初始字符, 其它大写字母为非终结符, 小写字母为终结符)

- $|w| = 1$, 总是 $S \rightarrow a$ 形式的 1 步生成, 成立.
- 假设 $|w| = k \leq n$ 时 w 需要 $2k - 1$ 次重写生成.

由于 $n + 1 \geq 1$, 则第 1 次重写必定是 $S \rightarrow AB$ 形式的重写,

w.l.o.g. 设 A 最终生成了 w_a , B 最终生成了 w_b , $w = w_a w_b$, 则 $w_a \neq \epsilon$ 且 $w_b \neq \epsilon$, 因为乔姆斯基范式规定只有 S 能重写为 ϵ , 所以 $|w_a| \geq 1, |w_b| \geq 1$, 又 $|w_a| + |w_b| = n + 1$, 所以 $|w_a| \leq n, |w_b| \leq n$, 满足归纳假设,

所以 A 最终生成 w_a 需要 $2|w_a| - 1$ 次重写, B 最终生成 w_b 需要 $2|w_b| - 1$ 次重写, 即 S 生成 w 需要 $1 + 2|w_a| - 1 + 2|w_b| - 1 = 2[|w_a| + |w_b|] - 1 = 2|w| - 1$ 次重写.

根据强归纳法可知定理成立.

4 Homework 04–Context Free 2

Problem 1 (2.20) 记 $A/B = \{w|wx \in A, x \in B\}$, 证明: 若 A 是上下文无关语言, B 是正则语言, 则 A/B 是上下文无关语言.

解 设 PDA $P_A = (Q_A, \Sigma, \Gamma, \delta_A, q_A, F_A)$ 识别 A , DFA $D = (Q_B, \Sigma, \delta_B, q_B, F_B)$ 识别 B .
可构造 PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ 识别 A/B :

- $Q = Q_A \cup (Q_A \times Q_B), q = q_A, F = F_A \times F_B$;
- 对于 $q_1 \in Q_A, q_2 \in Q_B$, 记 top 为栈顶字符,

$$\delta(q_1, a, top) = \begin{cases} \delta_A(q_1, a, top) & a \neq \epsilon \\ \delta_A(q_1, \epsilon, top) \cup \{\langle q_1, q_B \rangle, \epsilon\} & a = \epsilon \end{cases}$$

$$\delta(\langle q_1, q_2 \rangle, a, top) = \begin{cases} \emptyset & a \neq \epsilon \\ \langle \delta_A(q_1, r, top), \delta_B(q_2, r) \rangle, \delta_A(q_1, r, top) & a = \epsilon, \forall r \in \Sigma \end{cases}$$

即 P 的行为是: 随机猜 B 中的字符串开始的位置, 并随机猜 x 的内容随机前进, 直到 P_A 和 D 都进入接受状态, 所以能够识别 A/B , 即证 A/B 是 CFL.

Problem 2 (2.30d) 根据泵引理, 证明: $D = \{t_1 \# t_2 \# t_3 \# \cdots \# t_k | k \geq 2, t_i \in \{a, b\}^*, (\exists i \neq j) t_i \neq t_j\}$ 不是 CFL.

解 $a^p b^p \# a^p b^p \in D$, 对于 CFL 泵引理中 $uvxyz$ 的形式, v 和 y 不能包含 $\#$, 否则 $uv^0 xy^0 z$ 不含 $\#$, 此时 $k = 1$ 不成立.

$\because |vxy| \leq p, \therefore vxy$ 只能是:

- vxy 均在 $\#$ 左侧或 vxy 均在 $\#$ 右侧, 此时 $uv^2 xy^2 z$ 必然导致 vxy 所在的一侧比另一侧长, 不成立;
- $vxy \subset b^p \# a^p$, 则必然有 $\# \in x$, 此时 $uv^2 xy^2 z$ 必然导致左侧 b 的个数多于 p 或右侧 a 的个数多于 p , 不成立;

因此 D 不可泵, 根据泵引理, D 不是 CFL.

Problem 3 (2.32) 证明: $\Sigma = \{1, 2, 3, 4\}$ 上的语言 $C = \{w \in \Sigma^* | w \text{ has the same number of 1 and 2, and the same number of 3 and 4}\}$ 不是 CFL.

解 $1^p 3^p 2^p 4^p \in C$, 由于 $|vxy| \leq p$, 所以 vxy 只能是 $1^p, 3^p, 2^p, 4^p, 1^p 3^p, 3^p 2^p, 2^p 4^p$ 其中之一的子串, $uv^0 xy^0 z$ 必定使该字符串长度减少, 而又不可能使得 1 和 2 的个数同时减少或 3 和 4 的个数同时减少, 因此 C 不可泵, 根据泵引理, C 不是 CFL.

Problem 4 (2.35) 设 G 是乔姆斯基范式的 CFG, 且设计 b 个不同变量, 证明: 若 G 生成某个串的过程用了至少 2^b 步, 则 $L(G)$ 是无限集.

解 由已知, G 生成该串的生成树中至少包含 2^b 个内结点.

因为满二叉树高度为 k 时包含 $2^k - 1$ 个结点, 因此该生成树的高度至少为 $b + 1$, 说明该路径上出现了重复的变量, 所以生成树可嵌套, $L(G)$ 无限.

Problem 5 (2.36) 参考 1.54 (Homework 02–Problem 4), 给出不属于 CFL 的语言, 其满足泵引理, 并证明二者.

解 $F = \{a^i b^j c^k d^l \mid i, j, k, l \geq 0, i = 1 \Rightarrow j = k = l\}$.

1. 证明: F 不是 CFL.

反证法: 假设 F 是 CFL, 根据 CFL 和正则语言的交集是 CFL, 有 $G = F \cap \{ab^i c^j d^k \mid i, j, k \geq 0\} = \{ab^i c^i d^i \mid i \geq 0\}$ 也是 CFL.

只要证明: G 不是 CFL, 可利用 G 不满足可泵性证明.

$ab^p c^p d^p \in G, \because |vxy| \leq p$, 且 v, y 均不能包含 a , 否则 $uv^0 xy^0 z$ 不包含 $a, uv^0 xy^0 z \notin G, \therefore v, y$ 只可能是 $b^p, c^p, d^p, b^p c^p, c^p d^p$ 其中之一的字串, $uv^0 xy^0 z$ 必定使该字符串长度减少, 而又不可能使得 b, c, d 的个数同时减少, 因此 G 不可泵, 不是 CFL.

根据反证法, F 也不是 CFL.

2. 证明: F 满足 CFL 的泵引理.

分类讨论如下:

- $i = 0, j = 0, k = 0, l = 0$ 不满足 $|uvxyz| > p$, 不符合泵引理的前提;
- $i = 0, j = 0, k = 0, l > 0$, 取 $v = x = \epsilon, y = d$ 即可泵;
- $i = 0, j = 0, k > 0$, 取 $v = x = \epsilon, y = c$ 即可泵;
- $i = 0, j > 0$, 取 $v = x = \epsilon, y = b$ 即可泵;
- $i = 1, j = k = l$, 取 $u = \epsilon, y = a$ 即可泵;
- $i = 2$, 取 $u = \epsilon, y = aa$ 即可泵, 因为此时 a 的个数必然为偶数, 不可能为 1, 对 b, c, d 的个数无要求, 符合 $a^i b^j c^k d^l$ 的格式即可;
- $i \geq 3$, 取 $u = \epsilon, y = a$ 即可泵, 因为此时 a 的个数 ≥ 2 , 不可能为 1, 对 b, c, d 的个数无要求, 符合 $a^i b^j c^k d^l$ 的格式即可;

所以 F 满足泵引理.

5 Homework 05–Church Turing Thesis

Problem 1 (3.6) 定理: 某个语言是可识别的当且仅当存在枚举器能枚举该语言中的串.

为什么以下证明充分性的过程有问题？

证明：设 TM M 识别语言 L ，令 TM $N\langle x \rangle$ ：忽略输入，有序枚举 $S_1, S_2, S_3, \dots \in \Sigma^*$ ，执行：模拟 $M\langle S_i \rangle$ ；若 M 接受 S_i ，则打印 S_i 。此时枚举器 N 能够枚举 L 中的串，即证。

解 若 L 中存在 w 使得 $M\langle w \rangle$ 无限循环，则 N 也会在 $M\langle w \rangle$ 处无限循环，不能枚举后面可能存在的字符，不成立。

Problem 2 (3.7)为什么以下描述不是合法的图灵机描述？

$M_{bad}\langle p \rangle$ ： p 是关于 $x_1, x_2, x_3, \dots, x_k$ 的多项式，枚举 $x_1, x_2, x_3, \dots, x_k$ 的整数取值，若使得 p 的值为 0 则接受，若所有取值均不能使得 p 的值为 0 则拒绝。

解 $x_1, x_2, x_3, \dots, x_k$ 的整数取值有无穷多种组合，与图灵机模型的状态集有限矛盾。

Problem 2 (3.15ac)证明：可判定语言在并集、克莱尼星号运算下封闭。

解 并集：设 TM M_1 可判定 L_1 ，TM M_2 可判定 L_2 ，令 TM $M\langle w \rangle$ ：

1. 运行 $M_1\langle w \rangle$ ，若结果为接受，则 M 接受 w ；
2. 运行 $M_2\langle w \rangle$ ，若结果为接受，则 M 接受 w ；
3. M 拒绝 w 。

因为所有步骤均可在有限时间内完成，所以 M 可判定 $L_1 \cup L_2$ ，即证。

克莱尼星号：设 TM M_0 可判定 L_0 ，令 TM $M\langle w \rangle$ ：

1. 利用隔板法列举 w 的所有分划 $w = w_1 w_2 w_3 \cdots w_k$ ，在某种分划下依次运行 $M_0\langle w_1 \rangle, M_0\langle w_2 \rangle, M_0\langle w_3 \rangle, \dots, M_0\langle w_n \rangle$ ，若所有子串的结果均为接受，则 M 接受 w ；
2. 若列举完所有分划后仍未接受，则 M 拒绝 w 。

因为输入的分划是有限的，而子串的判定也是有限的，所以 M 可判定 L_0^* ，即证。

Problem 4 (3.16ac)证明：可识别语言在并集、克莱尼星号运算下封闭。

解 并集：设 TM M_1 可判定 L_1 ，TM M_2 可判定 L_2 ，令 TM $M\langle w \rangle$ ：

1. 交替运行 $M_1\langle w \rangle, M_2\langle w \rangle$ ，若任意结果为接受，则 M 接受 w ；
2. M 拒绝 w 。

若 $w \in L_1 \cup L_2$ ，则有 $w \in L_1 \vee w \in L_2$ ，必然有 M_1 接受 w 或 M_2 接受 w ，此时 M 接受 w ，所以 M 可识别 $L_1 \cup L_2$ ，即证。

克莱尼星号：设 TM M_0 可判定 L_0 ，令 TM $M\langle w \rangle$ ：

1. 利用隔板法列举 w 的所有分划;
2. 交替运行全部分划下的每个子串 $M\langle w_i \rangle$, 若某种分划下所有子串的结果均为接受, 则 M 接受 w ;
3. M 拒绝 w .

若 $w \in L_0^*$, 则存在 w 的某种分划 $w = w_1 w_2 w_3 \cdots w_k$ 使得 $\{w_1, w_2, w_3, \dots, w_k\} \subseteq L_0$, 对应 $M\langle w_i \rangle$ 的运行结果均为接受, 此时 M 接受 w , 所以 M 可识别 L_0^* , 即证.

Problem 5 (3.22) 设语言 $A = \{s\}$,

$$s = \begin{cases} 0, & \text{there is not life on Mars} \\ 1, & \text{there is life on Mars} \end{cases}$$

A 是否可判定? 为什么?

解 可判定. 原因如下:

- $A = \{0\}$ 或 $A = \{1\}$ 是有限集, 有限集必然可判定.
- 火星上是否有生命仍然未知, 所以不能给出判定 A 的图灵机的描述, 但可以给出 2 个图灵机的描述, 其中必然有且仅有 1 个可判定 A .

6 Homework 06–Decidability

Problem 1 (4.5) $E_{TM} = \{\langle M \rangle \mid M \text{ is TM}, L(M) = \emptyset\}$, 证明: $\overline{E_{TM}}$ 是可识别的.

解 令 TM $P\langle M \rangle$:

1. 初始令 $i = 0$;
2. 有序枚举 Σ_M^* 中所有长度为 i 的串 $w_{i1}, w_{i2}, w_{i3}, \dots, w_{i\Sigma_M^i}$, 将所枚举的字符串加入已枚举字符串集;
3. 对 $\forall j \leq i, k = 1, 2, 3, \dots, |\Sigma_M|^j$, 模拟 $M\langle w_{jk} \rangle$ 运行有限步 (对于新加入已枚举字符串集的串, 从初始状态开始模拟; 否则从上次运行到的状态开始模拟);
4. 若任意运行达到了接受状态, 则 P 接受 M ; 若达到了本次模拟的步数上限, 则暂存记录状态, 运行下个待模拟的内容;
5. 将 i 自增, 回到第 2 步.

若 $\langle M \rangle \in \overline{E_{TM}}$, 则 $\exists w \in \Sigma_M^*, M$ 接受 w , 此时必然能在 P 中 $i = |w|$ 时被枚举到, 并在有限时间内运行到 M 的接受状态, 所以 P 可识别 $\overline{E_{TM}}$, 即证.

Problem 2 (4.11) $INF_{PDA} = \{\langle P \rangle \mid P \text{ is PDA}, |L(M)| = \infty\}$, 证明: INF_{PDA} 是可判定的.

解 可将 PDA 转化为其能识别的 CFG, 且可将 CFG 转化为等价的乔姆斯基范式, 将乔姆斯基范式中的变量构造为结点 (在以下证明过程中, 设 S 为初始字符, 其它大写字母为非终结符, 小写字母为终结符),

- 对于 $S \rightarrow AB$ 形式的重写规则, 添加边 $S \rightarrow A$ 和 $S \rightarrow B$;
- 对于 $A \rightarrow a$ 形式的重写规则, 添加边 $A \rightarrow a$;

此时, $|L(P)| = \infty$ 当且仅当图中有回路; 之后, 由于有向图中有无回路问题是可判定的, 所以 INF_{PDA} 问题也是可判定的, 即证.

Problem 3 (4.20) 设语言 A 和语言 B 不相交, $A \subseteq C, B \subseteq \bar{C}$, 称 C 能够分离 A 和 B . 证明: 若 A 和 B 是补可识别的, 则存在可判定的语言 C 能够分离 A 和 B .

解 $A \cap B = \emptyset \Rightarrow \overline{A \cap B} = \overline{A} \cup \overline{B} = \Omega$

由于 A 和 B 都是补可识别的, 所以 $\exists \text{ TM } M$ 可识别 A , $\exists \text{ TM } N$ 可识别 B .

对于 $w \in \{\Sigma_A \cup \Sigma_B\}^*$, 设 $\text{TM } P$ 识别语言 C , 令 $\text{TM } P\langle w \rangle$:

1. 交替运行 $M\langle w \rangle$ 和 $N\langle w \rangle$ 有限步, 直到 M 接受 w 或 N 接受 w ;
2. 若 M 接受 w , 则 P 接受 w ; 若 N 接受 w , 则 P 拒绝 w .

分类讨论其可行性如下: 由于 A 与 B 不相交, 所以不存在 $w \in A \wedge w \in B$;

- $w \in A$, 则必然有 $w \notin B$, 即 $w \notin \bar{A} \wedge w \in \bar{B}$, 则只有 N 会接受 w , 此时 $w \in \bar{C}$;
- $w \in B$, 则必然有 $w \notin A$, 即 $w \in \bar{A} \wedge w \notin \bar{B}$, 则只有 M 会接受 w , 此时 $w \in C$;
- $w \notin A \wedge w \notin B$, 即 $w \in \bar{A} \wedge w \in \bar{B}$, 则 M 和 N 均会接受 w , 此时 $w \in C$ 的真假未知, 但 P 必然会停止, 且与 A, C 或 B, C 之间的子集关系无关.

综上所述, P 必然能停止, 所以 C 是可判定的; 又有 $A \subseteq \bar{C}, B \subseteq C$, 令 $D = \bar{C}$, 则 D 也是可判定的, 且 $A \subseteq D, B \subseteq \bar{D}$, 所以 D 能够分离 A 和 B , 即证.

Problem 4 (4.24) 定义 PDA P 的无用状态为: $\forall w \in \Sigma^*, P\langle w \rangle$ 的运行过程不包含该状态. 证明: $L = \{\langle P \rangle \mid P \text{ is PDA}, P \text{ contains some useless states}\}$ 是可判定的.

解 令 $\text{TM } M\langle P \rangle$:

1. 枚举 P 的所有状态, 每枚举到 1 个状态, 就将其设为唯一接受状态, 并对新得到的 PDA 运行判空算法, 判断其识别的语言是否为空集, 直到得到识别语言为空的 PDA 或已枚举所有状态;
2. 若得到的某个 PDA 识别语言为空, 则 M 接受 P , 否则 M 拒绝 P .

由于 PDA 的状态有限, 且 PDA 识别语言为空可判定, 所以 M 可识别 L , 即证.

Problem 5 (4.26) 设 $PAL_{DFA} = \{\langle M \rangle \mid M \text{ is DFA, } M \text{ accepts some palindromic string}\}$.
证明: M 是可判定的.

解 w.l.o.g. 设 $\Sigma = \{0, 1\}$, 给出 CFG 的描述: $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$, 则该 CFG 生成回文串, 因此回文串的集合是 CFL. 又 M 识别的语言是正则语言, 二者交集为 CFL, 可构造对应的 PDA 识别二者交集, 判定该 PDA 是否为空即可判定 PAL_{DFA} , 即证.

7 Homework 07–Reducibility 1

Problem 1 (5.1) 证明: EQ_{CFG} 是不可判定的.

解 反证法: 假设 EQ_{CFG} 是可判定的, 则给出 CFG 的描述如下: $S \rightarrow 0S \mid 1S \mid \epsilon$, 则该 CFG 生成 $\{0, 1\}^*$, 通过 EQ_{CFG} 的判定算法比较任意 CFG (设为 G) 与上述 CFG 识别的语言是否相等, 即可实现 ALL_{CFG} 的可判定算法, 这与 ALL_{CFG} 的不可判定性矛盾, 因此 EQ_{CFG} 不可判定.

Problem 2 (5.2) 证明: EQ_{CFG} 是补可识别的.

解 只要证明 $\overline{EQ_{CFG}}$ 是可识别的.

设有 CFG G_1, G_2 , 令 TM $T\langle G_1, G_2 \rangle$:

1. 构造识别 G_1, G_2 的 PDA M_1, M_2 ;
2. 从 Σ^* 中枚举字符串, 每枚举到 1 个字符串 (设为 w), 就依次运行 $M_1\langle w \rangle, M_2\langle w \rangle$, 直到满足下列情况之一:
 - M_1 接受 w , 且 M_2 拒绝 w ;
 - M_1 拒绝 w , 且 M_2 接受 w ;
3. T 接受 $\langle G_1, G_2 \rangle$.

若 $L(G_1) \neq L(G_2)$, 则 $\exists w \in \Sigma^*, w \in L(G_1) \oplus w \in L(G_2)$, 而由于 PDA 的性质, T 处理每个字符串的时间是有限的, 必然能枚举到上述 w , 因此 $\overline{EQ_{CFG}}$ 可识别, 即证.

Problem 3 (5.11) 考虑向双带图灵机输入任意字符串时 (输入内容位于第 1 条带), 该双带图灵机向第 2 条带写入了字符. 证明该问题不可判定.

解 将 A_{TM} 问题中的 TM 改为: 接受输入之前向第 2 条带写入某个字符. 若上述问题可判定, 则 A_{TM} 也可判定, 矛盾, 因此上述问题不可判定.

Problem 4 (5.28) 证明 Rice 定理: 设 P 是图灵机模型的集合, 若 P 满足:

- P 是非奇异的, 即 $P \neq \emptyset \wedge P \neq \Sigma^*$;
- 若 $L(M_1) = L(M_2)$, 则 $\langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P$;

则 P 是不可判定的.

解 反证法: 设 P 可判定. 设 TM $T_\emptyset$ 满足 $L(T_\emptyset) = \emptyset$, 则必然有 $\langle T_\emptyset \rangle \notin P$, 否则令 P 只包含接受语言为空的 TM, 则 $\forall M, L(M) = \emptyset \Leftrightarrow \langle M \rangle \in P$, 此时 E_{TM} 问题可判定, 与其不可判定性矛盾, 所以 $\langle T_\emptyset \rangle \notin P$.

由 P 的非奇异性及 $\langle T_\emptyset \rangle \notin P$, 设 TM T_α 满足 $L(T_\alpha) \neq \emptyset$, 则 $\exists T_\alpha, \langle T_\alpha \rangle \in P$.

设 $\langle M, w \rangle$ 是 A_{TM} 问题中的对象, $x \in \Sigma_T^*$, 根据 $\langle M, w \rangle$, 令 TM $S_{M,w}\langle x \rangle$:

1. 根据 $\langle M, w \rangle$ 的信息, 模拟 $M\langle w \rangle$;
 2. 若 M 接受 w , 则运行 $T_\alpha\langle x \rangle$;
 3. 若 M 拒绝 w , 则运行 $T_\emptyset\langle x \rangle$;
- 若 $\langle M, w \rangle \in A_{TM}$, 此时 $L(S_{M,w}) = L(T_\alpha)$, 有 $\langle S_{M,w} \rangle \in P$;
 - 若 $\langle M, w \rangle \notin A_{TM}$, 此时 $L(S_{M,w}) = \emptyset = L(T_\emptyset)$, 有 $\langle S_{M,w} \rangle \notin P$;

综上所述, 若 P 是可判定的, 则 A_{TM} 也是可判定的, 矛盾, 所以 P 不可判定.

Problem 5 (5.30) 利用 Rice 定理, 证明: $\{\langle M \rangle \mid M \text{ is TM}, 1011 \in L(M)\}$ 不可判定.

解 设 $P = \{\langle M \rangle \mid M \text{ is TM}, 1011 \in L(M)\}$, 显然 $\exists M_1, 1011 \in L(M_1)$; $\exists M_2, 1011 \notin L(M_2)$, 所以 P 是非奇异的; 又若 $L(N_1) = L(N_2)$ 则 $1011 \in L(N_1) \Leftrightarrow 1011 \in L(N_2)$, 所以 $\langle N_1 \rangle \in P \Leftrightarrow \langle N_2 \rangle \in P$, 所以 P 满足 Rice 定理的条件, 不可判定.

8 Homework 08–Reducibility 2

Problem 1 (5.4) 若 $A \leq_m B$ 且 B 是正则语言, 能否说明 A 是正则语言? 为什么?

解 不能. 归约函数 f 是 $\forall w \in \Sigma_A^*, f(w) = 0$ 时, $\{0\}$ 是正则语言, A 可以是任何语言.

Problem 2 (5.7) 若 A 是可识别的, $A \leq_m \bar{A}$, 证明: A 是可判定的.

解 由于 A 是可识别的, 有 $\exists$ TM M , M 可识别 A ;

由于 $A \leq_m \bar{A}$, 有 $\exists f, \forall w \in \Sigma_A^*, f(w) \in \Sigma_A^*$ 可计算;

设 $w \in \Sigma_A^*$, 令 TM $N\langle w \rangle$:

1. 计算 $f(w)$;
 2. 轮询模拟 $M\langle w \rangle, M\langle f(w) \rangle$, 直到 M 接受 w 或 M 接受 $f(w)$;
 3. 若 M 接受 w , 则 N 接受 w ;
 4. 若 M 接受 $f(w)$, 则 N 拒绝 w .
 - 若 $w \in A$, 则 $f(w) \notin A$, M 会接受 w , N 接受 w ;
 - 若 $w \notin A$, 则 $f(w) \in A$, M 会接受 $f(w)$, N 拒绝 w ;
- 综上所述, N 可判定 A , 因此 A 是可判定的.

Problem 3 (5.22)证明: $A \leq_m A_{TM}$ 当且仅当 A 是可识别的.

解 证明: $A \leq_m A_{TM} \Rightarrow A$ 是可识别的.

由于 $A \leq_m A_{TM}$, 有 $\exists f, \forall w \in \Sigma_A^*, f(w) \in \Sigma_{A_{TM}}^*$ 可计算;

设 $x \in \Sigma_A^*$, 令 $TM T\langle x \rangle$:

1. 计算 $f(x) = \langle M, w \rangle$;
 2. 以 $\langle M, w \rangle$ 为输入, 运行 A_{TM} 的可识别算法, 并输出相同结果.
- 若 $x \in A$, 则 $f(x) \in A_{TM}$, T 接受 x , 因此 T 可识别 A , 即证.

证明: A 是可识别的 $\Rightarrow A \leq_m A_{TM}$.

由于 A 是可识别的, 有 $\exists TM M$, M 可识别 A ;

设 $w \in \Sigma_A^*$, 令 $f(w)$:

1. 输出 $\langle M, w \rangle$.
- 若 $w \in A$, 则显然 M 接受 w , $f(w) \in A_{TM}$, 因此 $A \leq_m A_{TM}$, 即证.

Problem 4 (5.23)证明: A 可判定当且仅当 $A \leq_m 0^*1^*$.

解 证明: A 是可判定的 $\Rightarrow A \leq_m 0^*1^*$.

由于 A 是可判定的, 有 $\exists \text{TM } M_A$, M_A 可识别 A ;

设 $w \in \Sigma_A^*$, 令 $f(w)$:

1. 运行 $M_A\langle w \rangle$;
 2. 若 M_A 接受 w , 输出 01;
 3. 若 M_A 拒绝 w , 输出 10.
- 若 $w \in A$, 则 M_A 接受 w , 此时 $f(w) = 01 \in \{0^*1^*\}$;
 - 若 $w \notin A$, 则 M_A 拒绝 w , 此时 $f(w) = 10 \notin \{0^*1^*\}$;
- 所以 $A \leq_m 0^*1^*$, 即证.

证明: $A \leq_m 0^*1^* \Rightarrow A$ 是可判定的.

由于 $A \leq_m 0^*1^*$, 有 $\exists f, \forall w \in \Sigma_A^*, f(w) \in \{0, 1\}^*$ 可计算;

易知 0^*1^* 是正则语言, 设有 DFA D , 且 $L(D) = \{w | w = 0^m 1^n, m \geq 0, n \geq 0\}$;

设 $w \in \Sigma_A^*$, 令 $\text{TM } T\langle w \rangle$:

1. 计算 $f(w) \in \{0, 1\}^*$;
 2. 运行 $D\langle f(w) \rangle$, 并输出相同结果.
- 若 $w \in A$, 则 $f(w) \in 0^*1^*$, D 接受 $f(w)$, T 接受 w ;
 - 若 $w \notin A$, 则 $f(w) \notin 0^*1^*$, D 拒绝 $f(w)$, T 拒绝 w ;
- 所以 T 可判定 A , 即 A 可判定, 即证.

Problem 5 (5.24, 有删改) 设 $J = \{w | (w = 0x \wedge x \in A_{TM}) \vee (w = 1y \wedge y \in \overline{A_{TM}})\}$, 证明: J 不可识别.

解 利用 $\overline{A_{TM}} \leq_m J$ 证明. 设 $w \in \Sigma_{A_{TM}}^*$, 令 $f(w) = 1w$, 则 f 可计算; 此时 $w \in \overline{A_{TM}} \Leftrightarrow 1w \in J$, 满足归约的性质, 因此 J 不可识别.

9 Homework 09—Time Complexity 1

Problem 1 (7.6) 证明: P 在并集、拼接、补操作下封闭.

解 **并集:** 设 DTM M_1 在 $\text{TIME}(O(n^x))$ 判定 L_1 , DTM M_2 在 $\text{TIME}(O(n^y))$ 判定 L_2 .
 设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 运行 $M_1\langle w \rangle$, 若 M_1 接受 w , 则 N 接受 w ;
2. 运行 $M_2\langle w \rangle$, 若 M_2 接受 w , 则 N 接受 w ;
3. N 拒绝 w .

时间复杂度: $O(n^x) + O(n^y) + O(1) = O(n^{\max\{x,y\}})$, 所以 $L_1 \cup L_2 \in P$.

拼接: 设 DTM M_1 在 $\text{TIME}(O(n^x))$ 判定 L_1 , DTM M_2 在 $\text{TIME}(O(n^y))$ 判定 L_2 .

设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 将 w 按照截断位置顺序, 分为 2 段 $w = w_1w_2$;
2. 运行 $M_1\langle w_1 \rangle$, 若 M_1 拒绝 w_1 , 则回到第 1 步;
3. 运行 $M_2\langle w_2 \rangle$, 若 M_2 拒绝 w_2 , 则回到第 1 步;
4. N 接受 w .

时间复杂度: $O(n)[O(n^x) + O(n^y)] + O(1) = O(n^{\max\{x,y\}+1})$, 所以 $L_1 \circ L_2 \in P$.

补: 设 DTM M 在 $\text{TIME}(O(n^k))$ 判定 L .

设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 运行 $M\langle w \rangle$, 并输出相反结果;

时间复杂度: $O(n^k)$, 所以 $\bar{L} \in P$.

Problem 2 (7.7)证明: NP 在并集、拼接操作下封闭.

解 并集: 设 NTM M_1 在 $\text{TIME}(O(n^x))$ 判定 L_1 , NTM M_2 在 $\text{TIME}(O(n^y))$ 判定 L_2 .

设 $w \in \Sigma^*$, 令 NTM $N\langle w \rangle$:

1. 随机假设 $w \in L_1$ 或 $w \in L_2$;
2. 利用 $M_1\langle w \rangle, M_2\langle w \rangle$ 验证上述假设.

时间复杂度: $O(1) + \max\{O(n^x), O(n^y)\} = O(n^{\max\{x,y\}})$, 所以 $L_1 \cup L_2 \in \text{NP}$.

拼接: 设 NTM M_1 在 $\text{TIME}(O(n^x))$ 判定 L_1 , NTM M_2 在 $\text{TIME}(O(n^y))$ 判定 L_2 .

设 $w \in \Sigma^*$, 令 NTM $N\langle w \rangle$:

1. 随机将 w 分为 2 段 $w = w_1w_2$;
2. 运行 $M_1\langle w_1 \rangle, M_2\langle w_2 \rangle$, 若结果均为接受, 则 N 接受 w .

时间复杂度: $O(n) + O(n^x) + O(n^y) + O(1) = O(n^{\max\{x,y\}})$, 所以 $L_1 \circ L_2 \in \text{NP}$.

Problem 3 (7.10)证明: $ALL_{DFA} \in P$.

解 忽略状态转移边的字符, 将 DFA 视为有向图, 对图进行任意遍历, 并检查遍历到的结点是否位于接受状态集, 若任意结点不属于接受状态集就拒绝, 若遍历完成则接受. 由于图遍历可在多项式时间内完成, 所以 $ALL_{DFA} \in P$.

Problem 4 (7.11)

(1) 证明 $EQ_{DFA} \in P$.

(2) 定义星闭语言 A 满足 $A = A^*$. 给出接受 DFA 为输入, 判断其识别的语言是否是星闭语言的算法 (注意: EQ_{NFA} 是否属于 P 未知).

解 (1) 设 DFA M , 多项式时间内可构造出识别语言 $\overline{L(M)}$ 的 DFA;

设 DFA M, N , 在 $O((|M||N|)^k)$ 时间内可构造出识别 $L(M) \cap L(N)$ 的 DFA, 根据均值不等式有 $|M||N| \leq \left(\frac{|M| + |N|}{2}\right)^2 \leq (\max\{|M|, |N|\})^2$, 因此也是多项式时间的;

因此, 对于 DFA A, B , 构造 DFA C, D , 使得 $L(C) = \overline{L(A) \cap L(B)}$, $L(D) = \overline{L(A)} \cap L(B)$; 之后分别将 C 和 D 作为输入, 运行 ALL_{DFA} 的算法, 若 $L(C) = L(D) = \Sigma^*$, 则 $L(A) \cap \overline{L(B)} = \overline{L(A)} \cap L(B) = \emptyset$, $[L(A) \cap \overline{L(B)}] = [\overline{L(A)} \cap L(B)] = \emptyset$, 即 $L(A) = L(B)$; 反之, 若 $L(C) \neq \Sigma^*$ 或 $L(D) \neq \Sigma^*$, 则 $L(A) \neq L(B)$;

由于以上步骤均可在多项式时间内完成, 因此 $EQ_{DFA} \in P$.

(2) 设 DFA D 识别的语言为 L , 以 $\langle D \rangle$ 作为输入进行如下算法:

1. 运行 $D(\epsilon)$, 若结果为拒绝, 则 L 不是星闭语言;
2. 将 D 的所有接受状态 $q_1, q_2, q_3, \dots, q_{|F|} \in F$, 分别将 q_i 设为初始状态, 用 (1) 中算法验证新 DFA 与原 DFA 是否等价, 若出现了不等价的情况则 L 不是星闭语言;
3. L 是星闭语言.

时间复杂度: 由于 (1) 中算法是多项式时间的, 将 (1) 中算法运行 $|F|$ 次仍是多项式时间, 所以上述算法仍是多项式时间的, 可在多项式时间内判定星闭语言.

10 Homework 10—Time Complexity 2

Problem 1 (7.12) 证明: 图同构问题属于 NP.

解 设图 G, H , 令 NTM $N\langle G, H \rangle$:

1. 验证 $|V(G)| = |V(H)|$. 若否, 则 N 拒绝 $\langle G, H \rangle$;
2. 对于 $V(G)$ 中的每个结点 v_{Gi} :
 - (a) 随机选择 $V(H)$ 中的 1 个结点 v_{Hj} ;
 - (b) 建立映射 $\langle v_{Gi}, v_{Hj} \rangle \in f$;
 - (c) 检查 v_{Hj} 是否在 f 中出现. 若是, 则 N 拒绝 $\langle G, H \rangle$;
3. 对于 $E(G)$ 中的每条边 $e_{Gi} = \{v_{Gi}, v_{Gj}\}$:
 - (a) 计算 $f(v_{Gi}), f(v_{Gj})$;
 - (b) 检查 $\langle f(v_{Gi}), f(v_{Gj}) \rangle$ 是否在 $E(H)$ 中出现. 若是, 则 N 拒绝 $\langle G, H \rangle$;

4. N 接受 $\langle G, H \rangle$.

所以图同构问题可在 NTM 上多项式时间内判定, 即证.

Problem 2 (7.16) 证明: NP 在克莱尼星号运算下封闭.

解 设 $L \in \text{NP}$, 且 NTM M 可在 $O(n^k)$ 时间内识别 L .

对于 $w \in \Sigma_L^*(|w| = n)$, 令 NTM $N\langle w \rangle$:

1. 若 $w = \epsilon$, 则 N 接受 w ;
2. 将 w 随机分为 k 段 $w = w_1 w_2 w_3 \cdots w_k$ (具体地, 将指针头置于 w 左侧, 指针头每次往右移动时, 就随机地插入或不插入隔板, 即可实现随机分割);
3. 运行 $M\langle w_1 \rangle, M\langle w_2 \rangle, M\langle w_3 \rangle, \dots, M\langle w_k \rangle$, 若任何运行结果为拒绝, 则 N 拒绝 w ;
4. N 接受 w .

时间复杂度: $O(n) + O(n)O(n^k) = O(n^{k+1})$ 仍然是多项式时间, 所以 NP 在克莱尼星号运算下封闭.

Problem 3 (7.18) 证明: 若 $\text{NP} = \text{P}$, 则 $\forall A \in \text{P}$ 且 A 非奇异, 有 $A \in \text{NP} - \text{C}$.

解 $\because \text{P} \subseteq \text{NP}, \therefore \forall A \in \text{P}, A \in \text{NP}$.

下面证明 $\text{SAT} \leq_P A$:

$\because \text{SAT} \in \text{NP} = \text{P}, \therefore \exists \text{DTM } L, L$ 可在多项式时间内判定 SAT 问题.

由于 A 非奇异, 必然存在 $x_{in} \in A$ 以及 $x_{out} \notin A$; 对于 $w \in \Sigma_{\text{SAT}}^*$:

1. 运行 $L\langle w \rangle$;
2. 若 L 接受 w , 则输出 x_{in} ;
3. 若 L 拒绝 w , 则输出 x_{out} ;

显然归约函数可在多项式时间内完成计算, $\therefore \text{SAT} \leq_P A$.

根据 NP 完全的性质, $\therefore \text{SAT} \in \text{NP} - \text{C}, \therefore A \in \text{NP} - \text{C}$.

Problem 4 (7.22) 设 $\text{DOUBLE} - \text{SAT} = \{\langle \phi \rangle \mid \phi \text{ has at least 2 satisfying assignments}\}$, 证明: $\text{DOUBLE} - \text{SAT} \in \text{NP} - \text{C}$.

解 先证明: $\text{DOUBLE} - \text{SAT} \in \text{NP}$.

设 $\langle \phi \rangle \in \Sigma_{\text{DOUBLE-SAT}}^*$, 令 NTM $M\langle \phi \rangle$:

1. 随机对 ϕ 中的变元赋值 2 次;
2. 验证 2 次赋值是否相同, 若是, 则 M 拒绝 $\langle \phi \rangle$;

3. 验证 2 次赋值是否均能满足 ϕ , 若否, 则 M 拒绝 $\langle \phi \rangle$;

4. M 接受 $\langle \phi \rangle$.

显然可在多项式时间内判定该问题.

再证明: $SAT \leq_P DOUBLE-SAT$.

设 $\langle \phi \rangle \in \Sigma_{DOUBLE-SAT}^*$, 令 $f(\phi) = \phi \wedge (x \vee \bar{x})$, 其中 x 未在 ϕ 中出现, 显然 f 可在多项式时间内计算 ϕ .

又有 $\langle \phi \rangle \in SAT \Leftrightarrow (\phi = True \wedge x = 0) \vee (\phi = True \wedge x = 1) \Leftrightarrow \langle \phi \wedge (x \vee \bar{x}) \rangle \in DOUBLE-SAT$, 所以 $DOUBLE-SAT \in NP-C$.

Problem 5 (7.24) 设 $CNF_k = \{\langle \phi \rangle \mid \phi \text{ can be satisfied, and each variable appears at most } k \text{ times.}\}$.

(1) 证明: $CNF_2 \in P$.

(2) 证明: $CNF_3 \in NP-C$.

解 (1) 对于 ϕ 中的任意变量 x ,

- 若其出现了 1 次, 则根据其出现的形式赋值 (即: 若析取子句中出现 x , 则赋值 $x = 1$; 若析取子句中出现 $\bar{x}$, 则赋值 $x = 0$), 即可使整个析取子式满足, 后续计算过程中可删去该析取子式;
- 若其出现了 2 次,
 - 出现在同个析取子句中时, 可根据其出现的形式赋值 (即: 若析取子句中出现 $x \vee x$, 则赋值 $x = 1$; 若析取子句中出现 $\bar{x} \vee \bar{x}$, 则赋值 $x = 0$; 若析取子句中出现 $x \vee \bar{x}$, 则该析取子式永真), 即可使整个析取子式满足, 后续计算过程中可删去该析取子式;
 - 出现在不同析取子句中 (设分别在 f_1, f_2 中出现) 时,
 - * 若 $x \in f_1, x \in f_2$, 则令 $x = 1$ 可使 f_1, f_2 同时满足, 后续计算过程中可删去该析取子式;
 - * 若 $\bar{x} \in f_1, \bar{x} \in f_2$, 则令 $x = 0$ 可使 f_1, f_2 同时满足, 后续计算过程中可删去该析取子式;
 - * 若 $x \in f_1, \bar{x} \in f_2$, w.l.o.g. 设 $f_1 = x \vee P, f_2 = \bar{x} \vee Q$, 根据归结证明法可归结得到 $P \vee Q$;

综上所述, 计算过程可依次处理每个变量, 使得化简得到的式中不包含该变量; 又由于不同变量的个数是 $O(|\phi|)$ 的, 可在多项式时间内处理所有变量, 因此 $CNF_2 \in P$.

(2) 先证明: $CNF_3 \in NP$.

设 $\langle \phi \rangle \in \Sigma_{CNF_3}^*$, 令 NTM $M(\phi)$:

1. 随机对 ϕ 中的变元赋值;

2. 验证赋值是否能满足 ϕ , 若否, 则 M 拒绝 $\langle \phi \rangle$;
3. M 接受 $\langle \phi \rangle$.

显然可在多项式时间内判定该问题.

再证明: $3-SAT \leq_P CNF_3$.

对于 $3-SAT$ 中出现 $m(m > 3)$ 次的变元 S , 设 S 所在的析取子式分别为 $S_1 \vee A_1 \vee B_1, S_2 \vee A_2 \vee B_2, S_3 \vee A_3 \vee B_3, \dots, S_m \vee A_m \vee B_m$, 其中 $S_i = S$ 或 $\bar{S}$, 替换为 $(T_1 \vee A_1 \vee B_1) \wedge (\bar{T}_1 \vee T_2) \wedge (T_2 \vee A_2 \vee B_2) \wedge (\bar{T}_2 \vee T_3) \wedge (T_3 \vee A_3 \vee B_3) \wedge (\bar{T}_3 \vee T_4) \wedge \dots \wedge (T_{m-1} \vee A_{m-1} \vee B_{m-1}) \wedge (\bar{T}_{m-1} \vee T_m) \wedge (T_m \vee A_m \vee B_m) \wedge (\bar{T}_m \vee T_1)$, 其中 $T_i = S_i$, 但 $i \neq j$ 时 T_i 与 T_j 是相互独立的不同变元, 则由于 $\bar{T}_i \vee T_j (i = 1, 2, 3, \dots, m, j = (i+1) \bmod m)$ 的限制, 有 $T_1 = T_2 = T_3 = \dots = T_m$, 保证 CNF_3 式的可满足性等价于 $3-SAT$ 式.

由于需要处理的变元数量是 $O(|\phi|)$ 的, 处理过程需要修改的变元和新插入的 $\bar{T}_i \vee T_j$ 也是 $O(|\phi|)$ 的, 所以可在多项式时间内完成归约.

$\therefore 3-SAT \leq_P CNF_3. \therefore CNF_3 \in NP-C.$

Problem 6 (7.40)证明: 若 $P = NP$, 则可在多项式时间内找到图 G 的最大团集.

解 $\because P = NP, \therefore SAT \in P. \because K-CLIQUE \leq_P SAT, \therefore K-CLIQUE \in P$. 依次对 G 运行 $|V(G)|-CLIQUE, (|V(G)|-1)-CLIQUE, (|V(G)|-2)-CLIQUE, \dots, 1-CLIQUE$ 算法可找出 G 的最大团集, 且执行多项式时间算法多项式次仍是多项式时间, 即证.

11 Homework 11—Space Complexity 1

Problem 1 (8.4)证明: PSPACE 在并、补、克莱尼星号运算下封闭.

解 并集: 设 DTM M_1 在 $O(n^x)$ 空间内判定 L_1 , DTM M_2 在 (n^y) 空间内判定 L_2 .

设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 运行 $M_1\langle w \rangle$, 若 M_1 接受 w , 则 N 接受 w , 否则清空条带;
2. 运行 $M_2\langle w \rangle$, 若 M_2 接受 w , 则 N 接受 w , 否则 N 拒绝 w .

相当于执行了多项式空间算法多项式次, 不改变多项式空间可判定性质, 即证.

补: 设 DTM M 在 $O(n^k)$ 空间内判定 L .

设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 运行 $M\langle w \rangle$, 并输出相反结果;

空间复杂度不变, 即证.

克莱尼星号: 设 DTM M 在 $O(n^k)$ 空间内判定 L .

设 $w \in \Sigma^*$, 令 DTM $N\langle w \rangle$:

1. 若 $w = \epsilon$, 则 N 接受 w ;
2. 利用隔板法, 固定 $O(|w|)$ 的空间, 用于列举 w 的所有分割 (具体而言, 在 w 的每相邻 2 个字符之间预留隔板空间; 开始时不插入任何隔板即分割 $w = w$, 每次列举下个分割时, 指针头从左到右寻找第 1 个未插入隔板的位置, 向其中插入隔板, 并将其之前的所有隔板去掉, 可得到未枚举过的分割; 直到所有位置均插入了隔板, 表示已枚举完所有分割), 设 w 的一个分割为 $w = w_1 w_2 w_3 \cdots w_m$; 若已枚举完所有可能的分割仍未接受, 则 N 拒绝 w ;
3. 依次运行 $M\langle w_1 \rangle, M\langle w_2 \rangle, M\langle w_3 \rangle, \dots, M\langle w_m \rangle$; 若结果均为接受, 则 N 接受 w ; 若任意结果为拒绝, 则回到第 2 步.

由于 $|w_1|, |w_2|, |w_3|, \dots, |w_m|$ 的长度均可视为 $O(|w|)$, 且有 $m = O(|w|)$, 所以 $N\langle w \rangle$ 最多需要 $O(|w| + m|w|^k) = O(|w|^{k+1})$ 空间, 不改变多项式空间可判定性质, 即证.

Problem 2 (8.8) 设 $EQ_{REG} = \{\langle R, S \rangle | R, S \text{ are equivalent regular expressions}\}$, 证明: $EQ_{REG} \in PSPACE$.

解 分别构造识别 R, S 的 NFA M_R, M_S , 设 $|Q_R|, |Q_S|$ 分别为 M_R, M_S 的状态集规模, F_R, F_S 分别为 M_R, M_S 的接受状态集;

构造 2 个状态集 C_R, C_S 分别表示 M_R, M_S 可能处于的状态, 设置计数值不超过 $2^{\max\{|Q_R|, |Q_S|\}}$ 计数器; 每次从 Σ^* 中随机选择 1 个字符分别输入到 M_R, M_S 上, 并依此更新 C_R, C_S , 之后验证 $(C_R \cap F_R = \emptyset) \leftrightarrow (C_S \cap F_S = \emptyset)$ 的真假: 若为真, 则计数器 +1 并重复此步骤, 直到计数器达到 $2^{\max\{|Q_R|, |Q_S|\}}$, 若仍为真则接受 $\langle R, S \rangle$; 若为假, 则拒绝 $\langle R, S \rangle$.

由于以上步骤可在 NTM 上多项式空间内完成, 所以 $EQ_{REG} \in NPSPACE$; 再根据 Savitch 定理 $PSPACE = NPSPACE$, 所以 $EQ_{REG} \in PSPACE$.

Problem 3 (8.14) *Cat - Mouse* 游戏定义为: 无向图 G 中存在某个结点 v 定义为 hole, 游戏开始时猫和鼠分别位于随机结点处, 双方轮行动, 每次行动猫和鼠分别可移动到当前位置的相邻结点; 若某次行动后, 猫和鼠位于相同结点, 则猫胜; 若某次行动后鼠位于 hole 结点处, 则鼠胜.

设 $CAT_{WIN} = \{\langle G, c, m, h \rangle | G \text{ is a graph, } c, m, h \text{ is cat's location, mouse's location, hole respectively, and if it's cat's turn, cat has winning strategy}\}$, 证明: $CAT_{WIN} \in P$.

解 对于给定的图 G ，游戏状态可用 $\{cat, mouse, turn\} (c \neq m, m \neq h)$ 表示，其中 $cat, mouse$ 分别为猫和鼠的位置， $turn = cat's$ 或 $mouse's$ 表示轮到猫或鼠行动。

定义 2 个状态集 $CATWIN, MOUSEWIN$ ，初始将所有满足 $v \neq h, \{u, v\} \in E(G)$ 的 $\{u, v, cat's\}$ 列入 $CATWIN$ ，将所有满足 $\{v, h\} \in E(G)$ 的 $\{u, v, mouse's\}$ 列入 $MOUSEWIN$ 。

遍历所有游戏状态，并在某轮遍历后没有状态改变时停止：

- 若访问到不属于 $CATWIN, MOUSEWIN$ 的 $\{u, v, cat's\}$ ，若 $\exists \{u, w\} \in E(G) : \{w, v, mouse's\} \in CATWIN$ ，则将 $\{u, v, cat's\}$ 列入 $CATWIN$ ；
- 若访问到不属于 $CATWIN, MOUSEWIN$ 的 $\{u, v, mouse's\}$ ，若 $\forall \{v, w\} \in E(G) : \{u, w, cat's\} \in CATWIN$ ，则将 $\{u, v, mouse's\}$ 列入 $CATWIN$ ；

最后检查 $\{c, m, cat's\}$ ，若 $\{c, m, cat's\}$ ，则 $\langle G, c, m, h \rangle \in CATWIN$

游戏状态数为 $2|V(G)|^2$ ，每次遍历至少处理 1 个状态，所以遍历次数不超过 $2|V(G)|^2$ ，处理过程不超过 $2|V(G)|^2$ ，因此算法可在多项式时间内完成，即证。

12 Homework 12—Space Complexity 2

Problem 1 (8.5)证明： $A_{DFA} \in LSPACE$ 。

解 在 TM 模拟 DFA 运行过程中，只需要记录已处理的长度 $O(\log |w|)$ 、DFA 当前状态 $O(\log |Q|)$ ，对 $\langle D, w \rangle$ 而言是对数空间的，即证。

Problem 2 (8.6)证明： $PSPACE - H \subseteq NP - H$ 。

解 设 $A \in PSPACE - H$ ，则 $\forall L \in PSPACE, L \leq_P A$ 。

$\therefore NP \subseteq PSPACE \quad \therefore \forall M \in NP, M \leq_P A \quad \therefore A \in NP - H$ 。

Problem 3 (8.11)若 $NP - H \subseteq PSPACE - H$ ，证明： $PSPACE = NP$ 。

解 参考8.6 (Problem 2)， $PSPACE - H \subseteq NP - H$ ，所以 $PSPACE - H = NP - H$ 。

$\therefore NP - C \subseteq NP - H \quad \therefore \forall A \in NP - C, A \in PSPACE - H$

$\therefore \forall L \in PSPACE, L \leq_P A \in NP \quad \therefore L \in NP \quad \therefore PSPACE \subseteq NP$

又 $NP \subseteq PSPACE \quad \therefore PSPACE = NP$ 。

13 Homework 13—Space Complexity 3

Problem 1 (8.20)证明: $MULT = \{a\#b\#c \mid a, b, c \text{ are binary numbers, and } c = a \times b\} \in L$.

解 在工作带上, 用对数空间 l_a, l_b, l_c 保存 a, b, c 的长度 $|a|, |b|, |c|$, 用对数空间保存指向 a, b, c 右数第 i 位的指针 p_a, p_b, p_c , 用 1 位 sum 保存待验证位的计算结果, 用对数空间缓存上个验证位进位 c_l 和当前验证位进位 c_t .

设 $w \in \Sigma_{MULT}^*$, 令 TM $M\langle w \rangle$:

1. 验证 $w = a\#b\#c$ 的形式, 若不符合, 则 M 拒绝 w ;
 2. 初始化 $p_a = p_b = p_c = 1, l_a = |a|, l_b = |b|, l_c = |c|$;
 3. 若 $p_a + p_b - 1 < p_c$:
 - (a) 若 $p_a < l_a$, 则令 p_a 增加 1, 并回到第 3 步;
 - (b) 若 $p_b < l_b$, 则令 p_b 增加 1, 并回到第 3 步;
 - (c) M 拒绝 w .
 4. 若 $p_a > 0$ 且 $p_b \leq l_b$:
 - (a) 计算 $a[p_a] \wedge b[p_b]$, 并将结果加在 sum 上, 若有进位则更新 c_t ;
 - (b) 令 p_a 减少 1, 令 p_b 增加 1, 并回到第 4 步;
 5. 用 c_l 更新 sum 和 c_t , 并将 sum 与 $c[p_c]$ 比较, 若不相同, 则 M 拒绝 w ;
 6. 若 $c_t = 0$ 且 $p_c = l_c$, 则 M 接受 w , 否则令 $p_a = 1, p_b = 1$ 后回到第 2 步.
- 因为总共使用了常数个对数空间, 所以 $MULT \in L$.

Problem 2 (8.25)

- (1)证明: 图是二分图当且仅当图中不包含奇数个结点的回路;
- (2)证明: $BIP_G = \{\langle G \rangle \mid G \text{ is bipartite graph}\} \in NL$.

解 (1)先证明: 二分图中不包含奇数个结点的回路.

设 $G = (U \cup V, E)$ 是二分图, 满足 $\forall \{v_i, v_j\} \in E, v_i \in U \leftrightarrow v_j \in V$, w.l.o.g. 设某个回路包含 $v \in V$, 则从 v 出发经过奇数边只能到达 U 中的结点, 不能形成回路, 即证.

再证明: 不包含奇数个结点的回路的图是二分图.

反证法: 假设 $G = (V, E)$ 不是二分图, 用 m 和 n 标记 V 中的结点, 设 $v \in V$, 将 v 标记为 m , 然后循环进行如下操作 (可用 BFS 实现), 直到所有结点均被标记:

1. 将标记为 m 的结点的所有未被标记的邻结点标记为 n ;
2. 将标记为 n 的结点的所有未被标记的邻结点标记为 m .

则 G 不是二分图当且仅当 $\exists \{u, v\} \in E$, x, y 标记相同, 此时从 v 出发到 x, y 的路径长必然相等 (否则与 BFS 的性质相矛盾), 则 v 到 x 与 v 到 y 路径长之和为偶数, 加上 $\{u, v\}$ 就得到了奇数个结点的回路, 由反证法即证.

(2) 设 $\langle G \rangle \in \Sigma_{\overline{BIP_G}}^*$, 令 TM $M\langle G \rangle$:

1. 保留 cnt 存储前进次数, 初始为 0;
2. 随机选择开始结点并记录, 同时记录当前结点, 初始为开始结点;
3. 随机选择与当前结点相邻的边, 并通过该边前进到相邻结点;
4. 若当前结点与开始结点相同, 且 cnt 为奇数, 则接受;
5. 令 cnt 增加 1, 若 $cnt = |V(G)|$ 则 M 拒绝 w , 否则回到第 3 步.

由于对数空间即可储存常数个结点以及不超过 $|V(G)|$ 的 cnt , 所以 $\overline{BIP_G} \in \text{NL}$.

又因为 $\text{NL} = \text{co-NL}$, 所以 $BIP_G \in \text{NL}$.

Problem 3 (8.30) 证明: $E_{DFA} \in \text{NL} - \text{C}$.

解 先证明: $E_{DFA} \in \text{NL}$.

设 $w = \langle Q, \Sigma, \delta, q_0, F \rangle \in \Sigma_{E_{DFA}}^*$, 令 TM $M\langle Q, \Sigma, \delta, q_0, F \rangle$:

1. 保留 cnt 存储前进次数, 初始为 0; 记录当前结点 q_t , 初始为 q_0 ;
2. 随机输入 1 个 Σ 中的字符, 更新 q_t ; 若 $q_t \in F$, 则 M 接受 w ;
3. 令 cnt 增加 1, 若 $cnt = |Q|$, 则 M 拒绝 w , 否则回到第 2 步.

由于对数空间即可储存当前个结点以及不超过 $|Q|$ 的 cnt , 所以 $\overline{E_{DFA}} \in \text{NL}$.

又因为 $\text{NL} = \text{co-NL}$, 所以 $E_{DFA} \in \text{NL}$.

再证明: $\overline{PATH} \leq_L E_{DFA}$.

设 $w = \langle G(V, E), s, t \rangle \in \Sigma_{PATH}^*$, 将 V 映射为 Q , 将 E 映射为 δ , 将 s 映射为 q_0 , 将 t 映射为唯一的 $q_{ac} \in F$. 由于 $\forall v \in V(G), d(v) \leq |V|$, 则可用 $O(|V|)$ 的 Σ 设定 δ 的输入参量, 此时用 $\log |w|$ 长度即可表示, 即证.

最后, 由于 $PATH \in \text{NL}$, 且 $\text{NL} = \text{co-NL}$, 所以 $E_{DFA} \in \text{NL} - \text{C}$.

Problem 4 (8.34) 设 $CYCLE = \{\langle G \rangle | G \text{ is directed graph, } G \text{ has some directed loops}\}$. 证明: $CYCLE \in \text{NL} - \text{C}$.

解 先证明: $CYCLE \in \text{NL}$.

设 $w = \langle G(V, E) \rangle \in \Sigma_{CYCLE}^*$, 令 TM $M\langle G \rangle$:

1. 保留 cnt 存储前进次数, 初始为 0; 记录当前结点, 初始为随机选择的开始结点;
2. 随机选择以当前结点为起点的边, 若不存在, 则 M 拒绝 w , 否则更新当前结点; 若当前结点与开始结点相同, 则 M 接受 w ;
3. 令 cnt 增加 1, 若 $cnt = |V|$, 则 M 拒绝 w , 否则回到第 2 步.

由于对数空间即可储存当前个结点以及不超过 $|V|$ 的 cnt , 所以 $CYCLE \in \text{NL}$.

再证明: $PATH \leq_L CYCLE$.

设 $w = \langle G(V, E), s, t \rangle \in \Sigma_{PATH}^*$, 构造 $G' = (V', E')$:

- $V' = \{v_i | v \in V, 0 \leq i \leq n\} \cup \{w\}$;
- 对 $\forall \{u, v\} \in E, i = 0, 1, 2, \dots, n-1$, 向 E' 中添加 (u_i, v_{i+1}) ;
- 对 $j = 1, 2, 3, \dots, n$, 向 E' 中添加 (t_j, w) ;
- 对 $k = 1, 2, 3, \dots, n$, 向 E' 中添加 (w, s_k) .

显然归约过程中, 单步操作只涉及常数个结点, 所以可在多数空间内归约.

最后, 由于 $PATH \in \text{NL}$, 所以 $CYCLE \in \text{NL} - \text{C}$.